Git

03 – Workflow

Question 1 - Compréhension

1. À quoi servent les commandes suivantes :

    - `git status`

Git status est une commande qui permet de voir le statut de l'arbre de travail actuelle, ce qui a été add par exemple. On peut donc, grâce à cette commande, voir les fichiers qui sont eu des modifications depuis le commit précédent, mais aussi les fichiers qui ne sont pas suivi (ceux dans le gitignore).

    - `git branch`

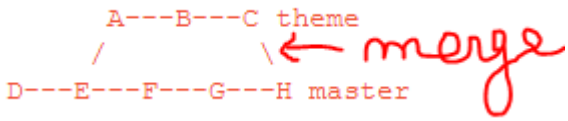
La commande git branch permet de créer, lister, renommer ou supprimer une branche. Pour lister il suffit de taper la commande git branch, pour en créer une nouvelle il faut faire git branch <nom de branche>. Enfin pour supprimer il faut rajouter un -d + le nom de la branche (`git branch (-d | -D) [-r] <nom-de-branche>…`).

    - `git checkout`

Cette commande permet de changer de branche de travail, mais aussi de restaurer
Il est aussi possible de changer de branche en en créant une nouvelle en même temps : ***`git checkout -b|-B <nouvelle-branche>`***. Et pour restaurer la branche : **`git checkout <commit> -- <fichier>`**

    - `git merge`

La commande git merge va permettre de rejoindre/fusionner plusieurs branches de travail ensemble.



- `git fetch`

La commande git fetch, permet de récupérer des branches depuis un autre dépôt, sans les intégrer directement dans la branche locale de travail.
Donc git fetch agit comme une mise à jour silencieuse.

1. Définir la notion de ***workflow***.

Le workflow est un ensemble de règles et de bonnes pratiques définies pour collaborer efficacement sur un projet. Cela inclut des étapes telles que la création de branches, la gestion des commits, et l'examen des modifications lors des push sur les branches de travail.
Souvent une personne (au moins) est responsable de vérifier les push et les merge pour garantir la cohérence et la stabilité du code.

2. Préciser les avantages de définir un *workflow* dans le cadre du développement d'un projet.

Définir un workflow permet une collaboration efficace, en évitant les redites et en maîtrisant la cohérence du code. C'est aussi une façon de gérer les conflits de code plus efficacement.

3. Expliquer pourquoi il est habituel de distinguer une branche `main` d'une branche `develop`.

Une branche develop est une branche de travail, à partir de laquelle vont être tirées d'autres branches de travail temporaire. Au contraire, la branche main est la branche qui est livrée au client, donc c'est la branche "définitive".

4. Expliquer le rôle du fichier `.gitignore`.

La fichier .gitignore est un fichier texte dans lequel l'on renseigne les fichiers ou dossiers qui ne doivent pas être pris en compte lors des commit. Cela permet d'exclure des fichiers temporaires ou des fichiers générés automatiquement par des outils (comme des fichiers le fichier vendor en twig).

5. Expliquer l'intérêt de définir un fichier `README.md` et préciser le format auquel l'extension `.md` fait référence.

Un fichier README.md va permettre de délivrer des informations tels que les objectifs du projet, comment installer le projet et les instances qui vont avec, mais aussi résumer le workflow.

(https://docs.github.com/fr/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes)
Le format .md fait référence à Markdown, un langage de balisage léger qui permet de formater du texte facilement. Les fichiers .md sont lisibles en brut mais aussi avec une mise en forme claire sur des plateformes comme GitHub, et même sur vS Code.
(https://www.geeksforgeeks.org/what-is-readme-md-file/#markdown)

Question 3 - *Workflow*

C - Implémentation d'une fonctionnalité : conclusion

- Fournir le lien vers votre dépôt GitHub (`03_git_questions.docx`) :

  https://github.com/TatianaAll?tab=repositories

- Si vous avez bien saisi l'idée de *« routine »*, il suffit simplement de revenir au point de départ : expliquer pourquoi.

Je ne comprends pas bien la question mais je vais tenter d'y répondre au mieux.
Une routine est la répétition des mêmes commandes git à faire de façon régulière (par exemple commit de façon régulière ou en faisant un git status souvent afin de voir les fichiers modifiés). Cela permet de garder une vision globale et à jour du projet.
Cela permet aussi d'avoir un code unifié avec des noms de branches homogène.
Cette pratique garantit la stabilité du projet et un contrôle continu sur l'évolution du code.